

Accelerating RL Post-Training Rollouts via System-Integrated Speculative Decoding

생각의 품질은 그대로, 기다림만 줄이는 법

Iso et al. · arXiv:2604.26779 · 2026.04.29

NeMo-RL × vLLM × MegatronLM × GRPO



AI는 답을 배우기 전에, 수많은 답을 말해 본다

Reasoning RL에서 학습 데이터는 미리 저장되어 있지 않다. 현재 모델이 직접 답안을 생성하고, 채점받고, 다시 배운다.

01 · PROMPT

문제를 받는다

수학 문제, 코드 과제, 도구 사용 요청



02 · ROLLOUT

여러 답을 끝까지 말한다

긴 사고 과정 × 여러 후보 = 가장 비싼 단계



03 · REWARD

정답을 채점한다

규칙, 테스트, verifier, 환경 보상



04 · UPDATE

더 나은 답을 학습한다

GRPO가 좋은 시도를 더 가능하게 만든다

“학습이 느린 이유는 역전파만이 아니라, 모델이 연습 답안을 한 토큰씩 말하기 때문이다.”

이 논문이 좋은 이유는 네 개의 시스템

을 한 장에 연결하기 때문이다

NeMo-RL

전체 RL 파이프라인과 Ray 기반
오케스트레이션

vLLM

빠른 rollout 생성과 speculative
verification

MegatronLM

분산 policy 학습과 verifier log-
prob 재계산

GRPO

그룹 보상으로 policy를 업데이트
하는 학습 규칙



한 RL step의 65~72%가 생성에 쓰였다

8B reasoning workload의 autoregressive baseline. backward를 최적화해도 rollout을 그대로 두면 전체 속도는 크게 줄지 않는다.

RL-Think · 185.3초

Generation 72%



RL-Zero · 151.2초

Generation 66%



■ Generation ■ Log-prob ■ Training ■ Data + Prepare

병목의 모양이 바뀌었다

Reasoning 모델은 더 긴 답을 생성한다.

GRPO는 한 prompt당 여러 답을 생성한다.

Agent는 한 trajectory에서 모델을 여러 번 호출한다.

결국 “얼마나 빨리 생각을 생성하는가”가 학습 속도를 지배한다.

빠르기만 해서는 안 된다 — 학습 할 분포가 바뀌면 다른 문제가 된다

$$\text{Learning speed} = \text{Effectiveness} \times \text{Throughput}$$

Effectiveness: 한 rollout이 주는 학습 신호의 가치

Throughput: 단위 시간당 처리하는 rollout과 update

Async / replay

일을 겹치거나 과거 trajectory를 재사용하지만 policy lag-off-policy 보정이 필요하다.

Low precision rollout

FP8 등으로 throughput을 높이지만 trainer와 rollout 분포의 mismatch를 관리해야 한다.

Speculative decoding

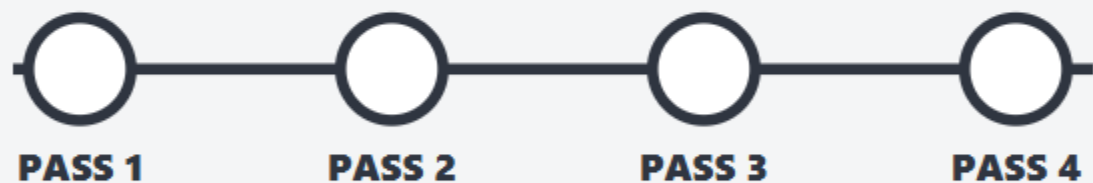
Verifier의 rejection sampling으로 최종 표본 분포를 그대로 보존한다.

Speculative decoding은 '초안 작성자 + 최종 편집자' 방식이다

Autoregressive · 한 토큰씩

큰 policy가 매번 전체 forward pass를 수행한다.

정답은 42 입니다 .



Speculative · 여러 토큰 제안

작은 draft가 제안하고, 큰 policy가 한 번에 검증한다.

정답은 ✓ 42 ✓ ~~아다 ✕~~ 입니다



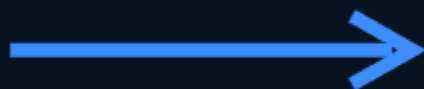
Draft를 믿는 것이 아니라, 수학적으로 교정한다

제안 토큰 x 를 받아들일 확률

$$a(x) = \min \left(1, \frac{p_{\theta}(x)}{q\varphi(x)} \right)$$

거절되면 남은 확률에서 다시 표본

$$r(x) \propto [p_{\theta}(x) - q\varphi(x)]_+$$



최종 토큰 $y \sim p_{\theta} \cdot \text{verifier policy}$ 와 동일한 분포

$p_{\theta} \cdot \text{Verifier}$

현재 RL policy. 최종 trajectory의 분포를 결정한다.

$q\varphi \cdot \text{Draft}$

빠른 제안자. 맞으면 시간을 벌고, 틀리면 교정된다.

결론

속도는 달라져도 GRPO가 보는 표본의 법칙은 같다.

RL에 넣는 순간, 'moving target' 문제가 생긴다

일반 serving에서는 target model이 고정되어 있다. RL에서는 policy가 매 step 바뀐다.

vLLM · Rollout engine

Draft가 k개 토큰 제안 → 현재 policy가 병렬 검증 → 정확한 rollout trajectory

반드시 최신 weight 필요

오래된 policy와 draft는 acceptance와 on-policy 의미를 동시에 해결할 수 있다.

결과

tokens, rewards, rollout metadata

Policy +
Draft
weights
↔
Rollouts

MegatronLM · Learner

현재 policy로 log-prob를 다시 계산하고 GRPO loss를 적용한다.

Policy path

GRPO gradient → θ update

Draft path

cached hidden states + detached policy logits → ϕ update

GRPO가 학습하는 것은 draft의 확률이 아니라 **verifier**의 확률이다

그룹 안에서 상대적으로 얼마나 좋았는가

$$\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \varepsilon}$$

현재 policy의 확률비로 좋은 답을 강화

$$L \approx -E [\min(\rho_t \hat{A}, \text{clip}(\rho_t) \hat{A})] + \beta \text{KL}$$

- 1 같은 prompt에서 여러 response를 생성한다.
- 2 reward를 그룹 평균·표준편차로 정규화한다.
- 3 **verifier policy**의 log-prob로 probability ratio를 계산한다.
- 4 draft loss는 별도로이며 policy gradient에서 분리된다.

한 문장 요약

Draft는 trajectory를 빨리 찾는 방법이고, GRPO의 학습 대상은 여전히 현재 policy π_θ 다.

Draft는 한 종류가 아니다 — MTP와 EAGLE-3는 다른 배치 경로를 제공한다

방법	어디에 붙는가	장점	주의점
MTP heads	본 모델 내부의 multi-token prediction head	별도 외부 모델 없이 native drafting	처음부터 MTP head가 있는 모델에 적합
EAGLE-3	policy hidden states를 쓰는 작은 외부 drafter	기존 pretrained model에도 일반적으로 적용	초기화·동기화·online adaptation 필요
Small draft LM	별도의 작은 언어모델	개념과 배포가 단순	tokenizer·distribution mismatch 관리
n-gram	최근 token 패턴을 재사용	학습 모델이 필요 없음	이 논문 실험에서는 acceptance가 있어도 더 느림

논문의 실험 선택

더 어려운 일반 경로인 EAGLE-3를 중심으로 검증했다.

시스템 결론

핵심 통합 원리는 MTP·외부 draft에도 공통으로 적용된다.

좋은 **drafter**는 한 번 만들고 끝나는 모델이 아니다

Policy가 학습하며 바뀌면 $q\varphi$ 는 점점 뒤쳐진다. 다만 이미 잘 맞는 draft라면 online update의 이득은 작을 수도 있다.



Offline draft

좋은 in-domain checkpoint를 고정해 generation에만 사용.

Online draft

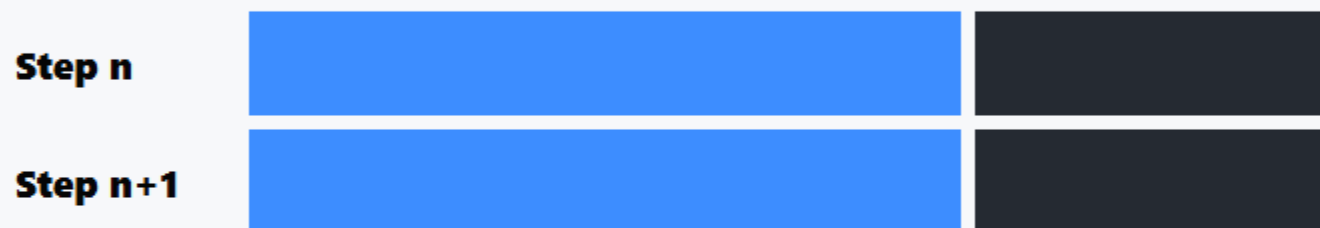
policy forward의 hidden state와 detached logits를 재 사용해 $q\varphi$ 를 업데이트.

$$L_{\text{total}} = L_{\text{GRPO}}(\theta) + \lambda \cdot L_{\text{draft}}(\varphi ; \text{stopgrad}(p\theta))$$

Synchronous는 기다리고, asynchronous는 겹친다

Synchronous RL

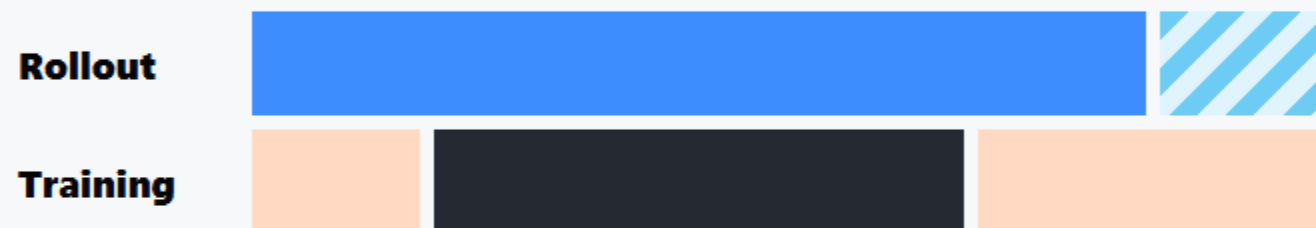
모든 rollout이 끝난 뒤 log-prob와 training 시작



SpecDec가 줄인 **rollout** 시간이 곧바로 **step latency** 감소로 이어진다.

Asynchronous RL

다음 rollout과 현재 training을 겹쳐 GPU idle을 숨김



이미 숨겨진 **generation**은 더 줄여도 **critical path**에 덜 반영된다.

1.35×

실측 sync · RL-Think end-to-end

1.41×

실측 sync · RL-Zero end-to-end

1.24×

실측 async · exposed idle 감소

실제 8B 실험에서 generation은
1.5~1.8x, 전체는 1.35~1.41x 빨라졌다

WORKLOAD	AR GENERATION	SPEC GENERATION	GEN SPEEDUP	AR OVERALL	SPEC OVERALL	E2E
RL-Think Qwen3-8B	133.6s	87.0s	1.54x	185.3s	137.4s	1.35x
RL-Zero Qwen3-8B-Base	100.0s	56.6s	1.77x	151.2s	107.5s	1.41x

32

GB200 GPUs · 실험 규모

k=3

기본 draft length

AIME

2024 validation accuracy · 변화 없음

해석: 1.8x는 “학습 전체”가 아니라 generation stage의 최고 측정치다. 나머지 단계가 전체 speedup의 천장을 만든다.

속도는 달라졌지만, 학습 곡선은 겹쳤다

RL-Think · AIME-2024

$\approx 0.60 \rightarrow \approx 0.70$



AR과 EAGLE-3 curve가 training 전체에서 밀접하게 겹침

RL-Zero · AIME-2024

$\approx 0.03 \rightarrow \approx 0.33$



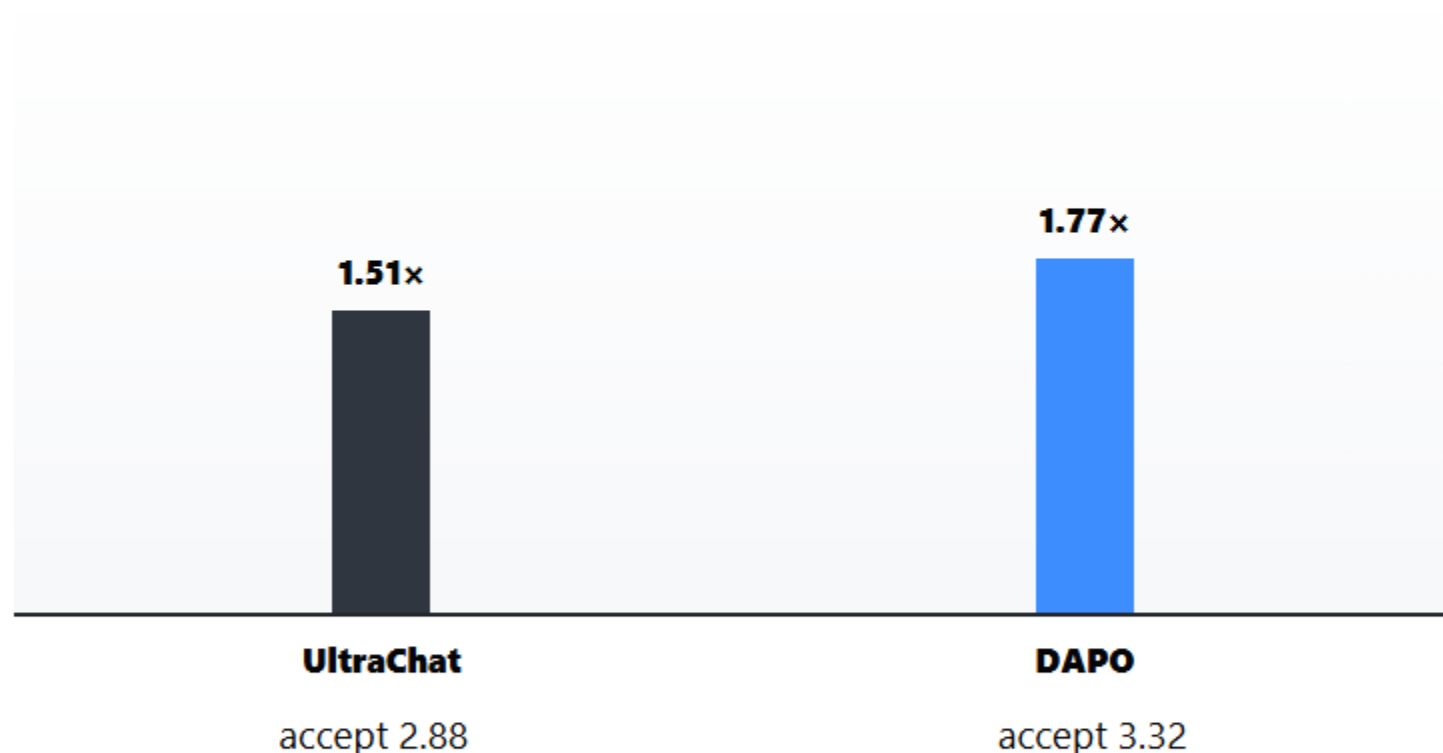
초기 약 100 step에서 답이 길어져도 speedup과 정확도 유지

논문의 품질 주장 범위: AIME-2024 validation trajectory가 구분되지 않았다. 모든 task에서 동일함을 증명한 것은 아니다.

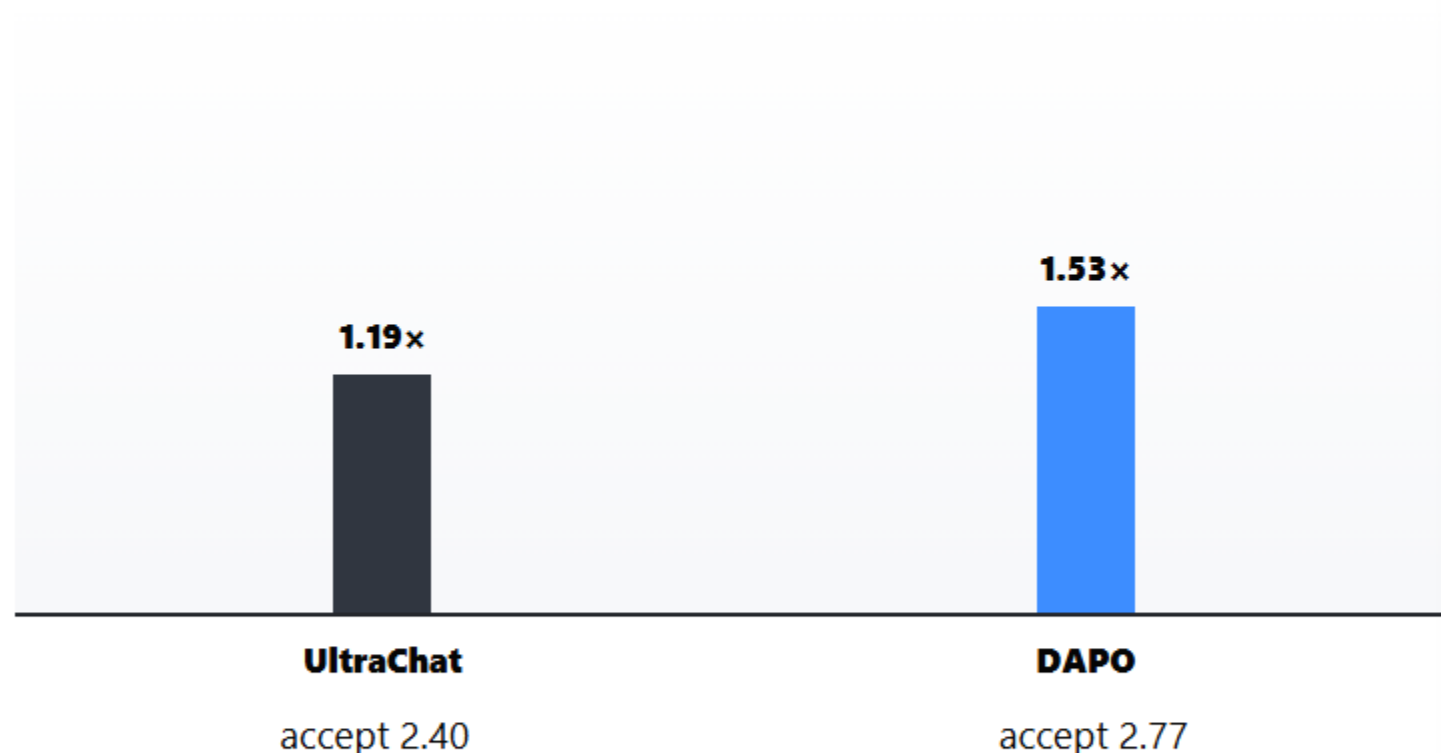
도메인에 맞는 draft가 generic chat draft보다 훨씬 낫다

같은 $k=3$ 이라도 "일반적으로 말을 잘 예측하는가"보다 "실제 rollout 분포와 맞는가"가 중요하다.

RL-Zero



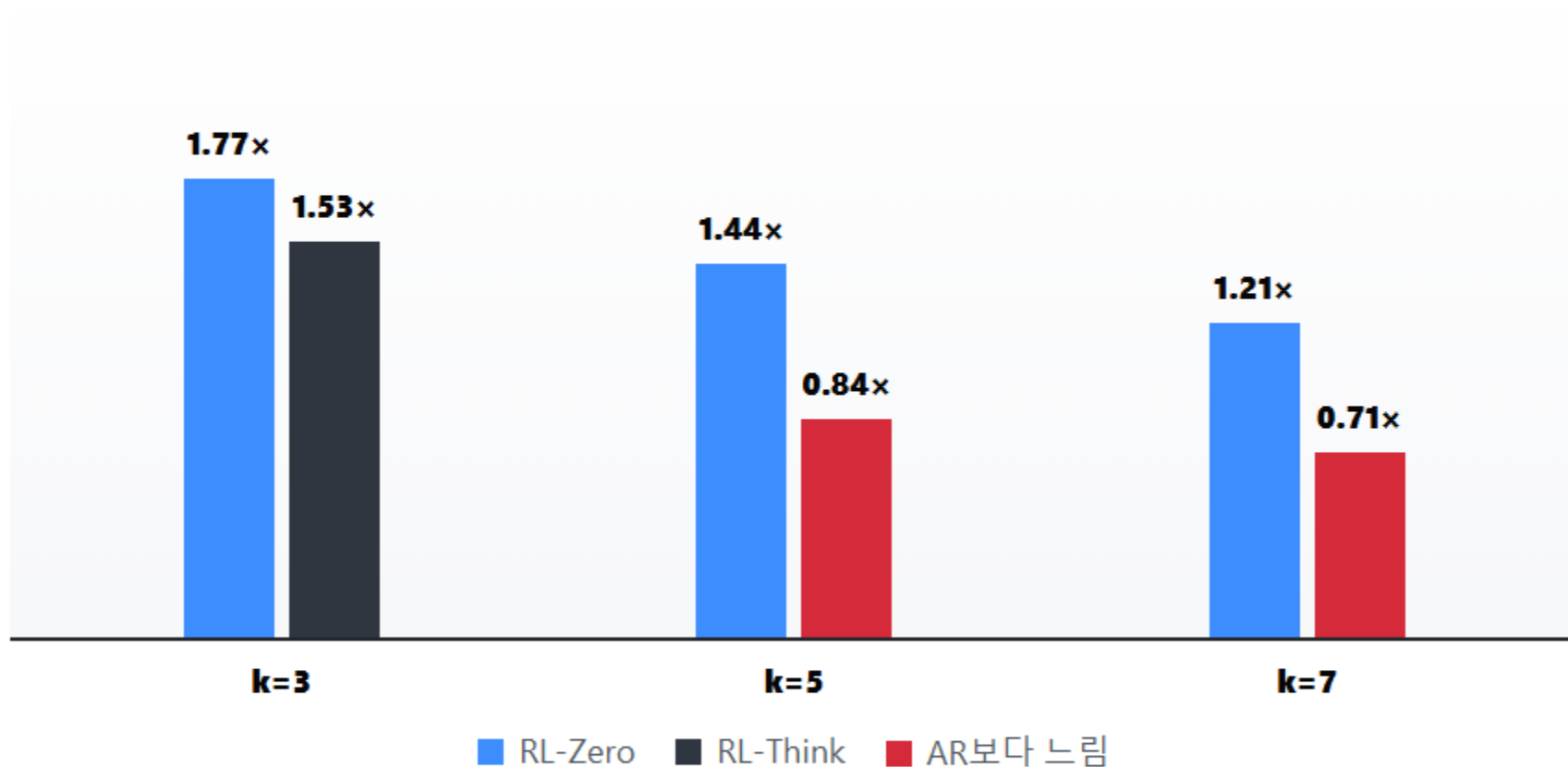
RL-Think



실무 교훈: draft pretraining 데이터는 배포 task의 prompt와 response 길이·스타일을 닮아야 한다.

더 많이 맞히는 것이 더 빠르다는 뜻은 아니다

Draft length k 에 따른 generation speedup



긴 draft의 역설

RL-Zero acceptance는 3.32 → 5.06으로 증가했지만 speedup은 1.77x → 1.21x로 하락.

n-gram의 반례

Acceptance 2.47인데 RL-Zero는 0.7x. RL-Think는 acceptance 2.05인데 0.5x.

측정된 **1.41x**와 전망된 **2.5x**는 같은 종류의 숫자가 아니다

MEASURED

1.35–1.41x

8B synchronous end-to-end

32 GB200 GPU 실험. Generation은 1.5–1.8x. 실제 software overhead 포함.

MEASURED

1.24x

8B asynchronous effective step

16-node non-colocated, policy lag 1. Exposed generation 10.4s → 0.6s.

PROJECTED

≈2.5x

235B frontier-scale end-to-end

Qwen3-235B-A22B, 2048 GB200, lag 2의 유리한 지점. Proprietary simulator 기반.

저자도 명시: 시뮬레이션 수치는 절대값이 아니라 “opportunity envelope”와 경향으로 해석해야 한다.

2026년 7월, 아이디어는 논문을 넘어 **실제 옵션**이 되었다



2026.04.29

논문 공개

NeMo-RL + vLLM + MegatronLM 안에서 verifier-exact rollout acceleration 제시.



2026.04.30

NeMo-RL v0.6

Speculative decoding을 공식 기능으로 출시. External draft, EAGLE-3, MTP 지원.



2026.07

공식 EAGLE-3 guide

Offline과 online workflow, Megatron refit, draft loss, YAML recipe가 문서화 됨.



2026.07.26

생태계 확장

vLLM Speculators는 pretrained EAGLE-3를 배포하고, Vime은 online MTP training을 문서화.

지금 가능한 것

NeMo-RL에서 vLLM EAGLE-3 generation-only 또는 Megatron 기반 online draft training.

현재 제약

NeMo online EAGLE-3는 Megatron 필요·sequence packing 미지원.
Vime external draft online training은 WIP.

가장 잘 맞는 곳은 '길고, 반복되고, 검증 가능한' 작업이다

수학·과학 reasoning

긴 chain-of-thought, 여러 후보 풀이, 정답 verifier가 있는 RLVR.

코드·SWE agents

테스트 통과 보상, 긴 수정 과정, 반복되는 code/tool call rollout.

기업형 tool agents

CLI 자동화, 보안 triage, 데이터 분석처럼 성공 조건을 검사할 수 있는 workflow.

도입 전 체크

- ✓ Generation이 step time의 큰 비중인가?
- ✓ Decode-heavy하고 response가 긴가?
- ✓ In-domain draft가 평균 2~3개 이상 맞히는가?

먼저 다른 곳을 볼 때

- ✗ 짧은 답·prefill 중심 workload
- ✗ 큰 batch에서 draft overhead가 더 큼
- ✗ Async가 generation을 거의 전부 숨김

좋은 가속은 답을 바꾸지 않는다. 다음 가설에 더 빨리 도달하게 한다.

1. 병목을 바꿔 본다

Reasoning-agentic RL에서는 rollout generation이 학습 시간을 지배한다.

2. 정확성을 지킨다

Draft는 제안만 하고 verifier가 policy distribution을 끝까지 책임진다.

3. 숫자의 종류를 묻는다

8B의 1.41×는 측정, 235B의 2.5×는 유리한 조건의 projection이다.

우리는 더 많은 GPU를 더하는 대신, 같은 GPU가 덜 기다리게 만들 수 있다.